

Practical Problems: Strings and Lists

Learning Outcome Addressed

- Recognize, interpret, and write programs with nested loops.
- Recognize and interpret basic recursive functions.
- Translate simple recursive algorithms to Python functions.

This assignment allows you to apply what you have learned in this module about strings and lists. Follow the instructions below to complete problems one to thirteen. If you get stuck on a problem and are not making any progress, please reach out to the learning facilitator for assistance.

1. [5 pts] starts_with

Write the function **starts_with(s, t, start)** that takes in two strings and a non-negative integer. The function should return True if string s contains the string t, starting at index start, and False otherwise.

You may not use the built-in startswith method.

Hint: There is a one-line solution.

2. [5 pts] ends_with

Write the function **ends_with(s, t)** that takes in two strings and returns True if string s ends with string t, and False otherwise. You may not use the built-in endswith method.

For an extra challenge and exercise, implement the function without using the string slicing operation (though you can certainly use string indexing).

3. [5 pts] find

Write the function **find(s, t, start)** that takes in two strings and a non-negative integer. The function should return the first index in s where substring t is found, starting at index start. If t is not found, return -1. You may not use the built-in find method.

Hint: You may want to use the starts_with function that you have already implemented.

4. [5 pts] count

Write the function **count(s, t)** that takes in two strings and returns the number of non-overlapping occurrences of t in s. You may not use the built-in count method.

Hint: You may want to use the starts_with function that you have already implemented.

5. [10 pts] strip

Write the function **strip(s)** that takes in a string and returns the same string with any leading and trailing spaces removed. For this problem, we are only considering " " as a space. You may not use the built-in strip method.

6. [10 pts] num_lines

Write the function **num_lines(s)** that takes in a string s and returns the number of lines in s, ignoring all the whitespace characters at the beginning and end of s.

Hint: There is a one-line solution that involves calling two string methods.

7. [10 pts] `insert_element`

Write the function **`insert_element(L, i, el)`** that takes in a list, index, and element, and destructively inserts the element into the list at index `i`. If `i` is greater than or equal to `len(L)`, the function should act the same as `L.append(el)`.

Since this function is destructive, you should not use any list slicing or any other operation that makes a copy or sublist of `L`. You also may not use the built-in `insert` method, or any list methods other than `append`. Additionally, the function should not return anything since it's a destructive function.

8. [15 pts] `duplicate_col`

Write the function **`duplicate_col(L, index, n)`** that takes in a 2D list `L`, a column index, and a number, and destructively modifies `L` so that the specified column at index `i` is duplicated `n` times.

For example, suppose `L` is

```
[[1,2,3],
```

```
[4,5,6],
```

```
[7,8,9]],
```

`i` is 0, and `n` is 2. Then the list should become

```
[[1,1,1,2,3],
```

```
[4,4,4,5,6],
```

```
[7,7,7,8,9]].
```

Since this is a destructive function, you should avoid any operations that make copies of lists (like slicing or the plus operator). Instead use appropriate destructive list method(s). Additionally, it should not return anything since it's a destructive function.

9. [15 pts] `get_averages_from_csv`

In an example that we went over in video, we saw how to convert a string in CSV (comma-separated values) format into a 2-dimensional list, and then do some basic computations on the data. Typically, the first line of a CSV string consists of headers/titles describing the data that the string contains. The remaining lines contain the actual data. For example, a CSV string might look like this:

```
University,Number of Students,Tuition
Carnegie Mellon University,13961,76760
Stanford University,16914,78218
Harvard University,22947,75891
University of California Berkeley,45057,41528
```

Write a function named `get_averages_from_csv` that takes as input a CSV string as above (where the first row consists of headers), as well as a header string. The returned value is the average of the column specified by the header string.

For the example above, if the header is "Number of Students", the returned value would be 24719.75.

If the given input header does not match any of the columns, return None. If the column corresponding to the given input header contains non-numeric values, return None. You may assume that if a column contains a numeric value, then it consists of all numeric values (except for the header) and the values are non-negative integers.

Some of the code you write may be similar (or the same) as the example we have seen in the video. But we highly recommend that after studying that example, you try to implement this function yourself without looking at the provided code.

Here are some methods that may be useful.

String methods: `split`, `splitlines`, `isdigit`, `strip`

List methods: `append`, `index`

10. [10 pts] list_to_csv_str

There is a string method called `join`. First do some research and learn how this method works.

Now write the function **list_to_csv_str(L)** that takes in a 2D list and returns the string representation of L in CSV format. In an example that we went over in video, we saw how to convert a string in CSV format into a 2D list. So our task here is to do the opposite, and convert the list back to a string.

For example, if the list L is

```
[["Alice", "Bob", "Charlie"],
```

```
[1, 2, 3],
```

```
[4, 5, 6]],
```

then the string returned should be "Alice,Bob,Charlie\n1,2,3\n4,5,6".

(Recall that "\n" represents the newline character.)

Hint: Use the `join` method of strings.

11. [10 pts] atbash_encrypt

In the lecture, we saw the example of Caesar shift as a simple encryption mechanism. In this example, we will implement a minor variant of it called the Atbash cipher. In this scheme, we map each letter to its reverse so that the "a" maps to "z", "b" maps to "y", "c" maps to "x", and so on. We map capital letters similarly. Any non-letter character remains the same.

Write the function **atbash_encrypt(s)** that takes in a string message and returns the encrypted message according to the scheme described above. Your implementation will be quite similar to the Caesar shift example. We highly recommend that you first study the example carefully, and then implement this function without looking at the example.

12. [5 pts] merge

Write the function **merge** that takes as input two sorted lists of integers and returns a new sorted list containing all the integers from the input lists. In other

words, you are asked to merge the two sorted input lists into one sorted list. You should not modify the input lists in any way.

For example, if the input list are [1, 1, 4, 5, 7] and [2, 4, 6, 8], then the function should return [1, 1, 2, 4, 4, 5, 6, 7, 8].

You are not allowed to use any list methods except for `append`. Furthermore, the efficiency of your function is important. In particular, you should not merge the two lists and then sort the merged list. This is not the most efficient solution and does not make use of the fact that the input lists are already sorted.

As you insert elements to the new list one by one, you should ensure that it is sorted (i.e. once all the elements are inserted, you should be done).

13. [5 pts] `insert_list`

Write the function **`insert_list(L, i, K)`** that destructively inserts list `K` into list `L` at index `i`. Since this is a destructive function, you should not use list slicing or any other operation that makes a copy of the list.

This is a generalization of the `insert_element` function that you implemented in the homework. You may wish to use this function as a helper.